Playwright with Typescript

INTRODUCTION

-Playwright is a open source automation framework created by Microsoft for web application testing

-It is designed to help you write tests that simulate real user interactions across different browser and devices

-Microsoft created playwright in 2020.

DEFAULT DEPENDENCIES:

1-Browser Supported-

- Chromium [Chrome Edge]
- Firefox
- Webkit [Safari] (for IOS)

2-Applications Supported-

- Web browser application
- Mobile Web application
- API

NOTE:
We can also check native aaplication but for that we have to use external plugin like Appium

3-Languages Supported-

- Typescript
- Typescript
- Python
- C#
- Java

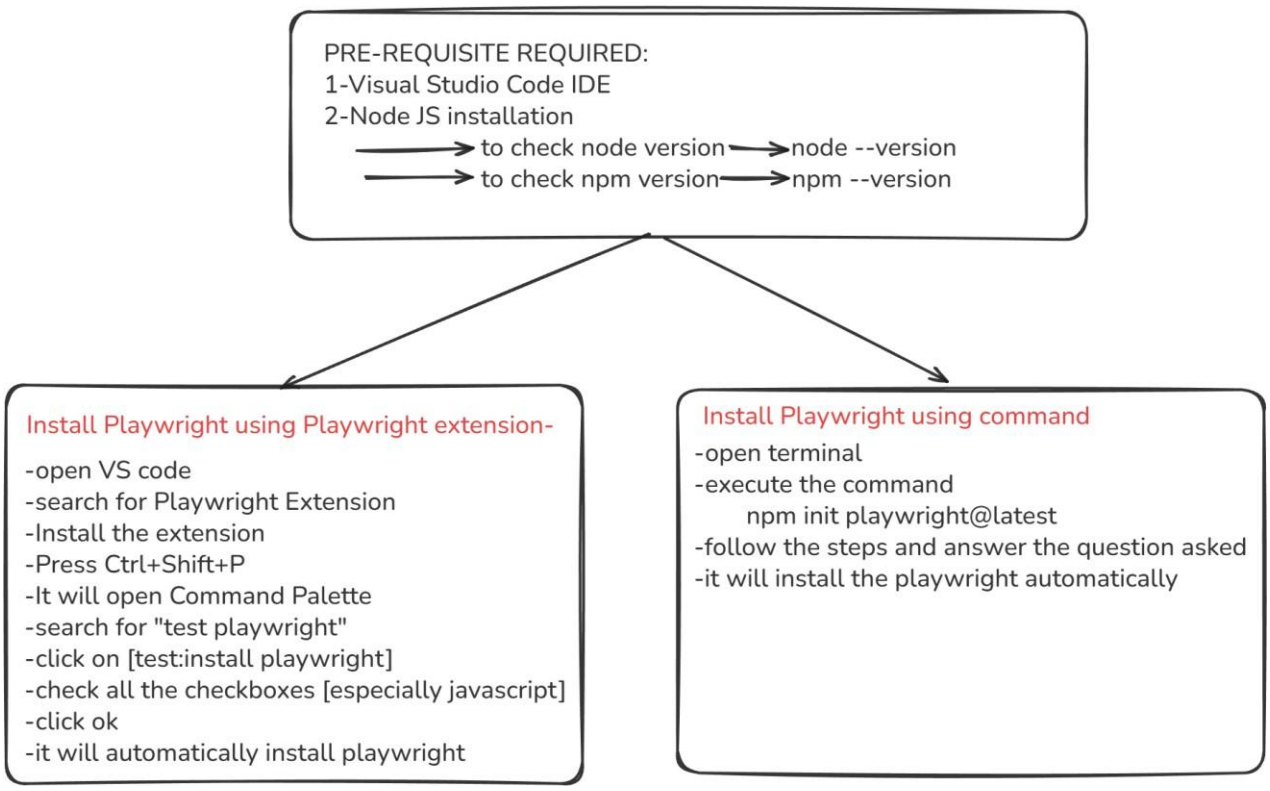
4-OS Support-

- Windows
- Linux
- MacOS
- Also supports CI runs [Jenkins]

ADVANTAGES:

1. Auto waiting
2. Tracing and Debugging
3. Built-in test runner/framework
4. Built-in reporter , Custom reports [HTML reporter]
5. Built-in assertions
6. Supports various types of Testing

INSTALLATION:



NPM:

- It is a tool and a package
- As a tool, it lets you install and manage packages

- As a repository ,it hosts millions of open source JS packages

NOTE:
When you install node Js , npm will be automatically installed

Default Folder Structure in Playwright

After installing Playwright in a project, Playwright automatically creates a standard folder structure. This structure helps organize test cases, configuration, reports, and test results in a clean and maintainable way.

---

tests/ Folder

The tests folder is where all Playwright test files are written.
Each test file usually contains test cases related to a specific feature or module of the application.

This is the main working folder for automation engineers.

---

playwright.config.ts / playwright.config.ts

This is the configuration file for Playwright.
It controls how tests are executed.

It is used to configure:

- Browser name
- Base URL
- Timeout values
- Viewport size
- Parallel execution
- Reports
- Headless or headed mode

Any global setting for the test run is managed from this file.

---

node_modules/ Folder

This folder contains all installed dependencies, including Playwright and other required packages.

- It is automatically created by npm
- It should not be edited manually
- Usually excluded from version control

---

package.tson File

This file contains project metadata and dependencies.

It includes:

- Project name and version
- Installed dependencies
- Playwright test commands (scripts)

Playwright test execution commands are usually added here.

---

package-lock.tson File

This file locks the exact dependency versions used in the project.
It ensures consistency across different environments and machines.

---

test-results/ Folder

This folder stores test execution artifacts, such as:

- Screenshots
- Videos
- Traces (if enabled)

It is mainly used for debugging failed tests.

---

playwright-report/ Folder

This folder contains the **HTML test report** generated after test execution.

The report shows:

- Passed, failed, and skipped tests
- Execution time
- Error messages
- Screenshots and traces (if available)

This folder is especially useful for reviewing results and CI reporting.

HOW TO CREATE FILE :

- Open Vs code
- Create a new file
- With filename.spec.ts

.spec ⟶ for specification file

.ts ⟶ for Typescript file

- After creating file 1st step is to import test from playwright
  import {test} from "playwright/test"
- Once after importing ,create the test block
  test("title_of_the _test",async()=>{})
- Here in test block the
  1st argument ⟶ The test Title
  2nd argument ⟶ The asynchronous

ASYNC AND AWAIT IN TYPESCRIPT:

-Typescript is asynchronous in nature, which means it does not wait for time-consuming operations to complete. Tasks such as API calls, page loading, or data fetching take time, and Typescript continues executing other code instead of blocking the execution.

-To handle such operations in a readable way, Typescript provides async and await.

- async is used to define an asynchronous function.
- An async function always returns a Promise.

-The await keyword is used inside an async function only. It tells Typescript to wait until the Promise is resolved before executing the next line of code. This waiting happens only inside the current function and does not block the entire program.

- await helps maintain proper execution order.
- It makes asynchronous code easy to read and understand.

-In Playwright, almost all actions such as launching the browser, navigating to a page, and interacting with elements are asynchronous. Therefore, the use of async and await is mandatory to ensure stable and reliable test execution.

FIXTURES:

- In Playwright ,fixture is a reusable piece of setup code that prepares something for your tests like opening a browser ,logging in ,creating data ,etc.

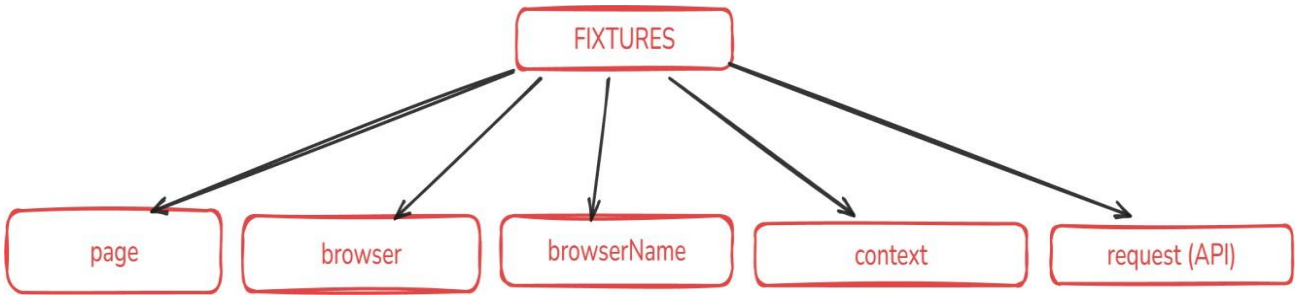
-Ready made object given by playwright with automatic setup and teardown

- Setup---- >creating browser/session/tabs
- Teardown ---->after test completes ,it close everything automatically

Why teardown is important--- >

1. The browser stays open
2. Ram consumptions increses
3. Performance gets low
4. Test will run slow
5. Sometimes it will fail the test because of the open resource

TYPES OF FIXTURES



PAGE FIXTURE:

-The page fixture represents a single browser tab or window. Playwright automatically creates a new browser context and a new page for every test and passes it to the test through this fixture. This means each test runs in an isolated environment.

- The page fixture is built into Playwright.
- It allows navigation, interaction, and validation on web pages.
- A fresh page is provided for each test execution.

-Using the page fixture improves test reliability by preventing shared state between tests. It also removes the need to manually launch the browser or create pages inside test cases.

-In Playwright automation, almost all user interactions such as clicking, typing, and assertions are performed using the page fixture, making it a core component of test scripts.

BROWSER FIXTURE:

In Playwright, the **browser fixture** represents the actual browser instance such as Chromium, Firefox, or WebKit. It is responsible for launching and managing the browser during test execution.

The browser fixture is provided by Playwright by default. It is created once and can be shared across multiple tests, while still maintaining isolation through separate browser contexts.

- The browser fixture controls the browser lifecycle.
- It supports Chromium, Firefox, and WebKit.
- It does not store user data or session information directly.

The browser fixture is mainly used to create **browser contexts**, which in turn create pages. Direct interaction with the UI is not performed using the browser fixture; instead, interactions happen through the page fixture.

Using the browser fixture improves performance by avoiding repeated browser launches and helps maintain a clean test architecture.

BROWSER NAME FIXTURES:

In Playwright, browserName is used to specify which browser should be used for test execution. It tells Playwright where the tests should run.

Playwright supports multiple browsers, and the browserName value is usually configured in the Playwright configuration file. Based on this value, Playwright launches the corresponding browser during test execution.

- chromium – Used for Chrome and Microsoft Edge
- firefox – Used for Mozilla Firefox
- webkit – Used for Safari (mainly for iOS testing)

Using browserName helps testers run the same test cases across different browsers without changing the test code. This makes cross-browser testing easy and efficient.

In real automation projects, browserName is commonly used to validate application behavior on multiple browsers and to run tests in parallel on different browser environments.

CONTEXT FIXTURES:

In Playwright, a **browser context** represents an isolated browser session. It acts like a fresh browser profile where cookies, cache, local storage, and session data are stored separately.

The **context fixture** is created from the browser fixture and is provided automatically by Playwright. Each test gets its own browser context, which ensures that tests do not share any data with each other.

- The context fixture provides test isolation.
- It stores session-level data such as cookies and local storage.
- Multiple pages can be created within a single context.

The context fixture sits between the browser and the page. The browser launches the context, and the context creates pages. This structure allows Playwright to run tests independently while still using the same browser instance.

Using context fixtures prevents test interference and improves reliability, especially when running tests in parallel or working with authenticated sessions.

Difference between page , context ,browser

- if u test a normal UI flow -----> take the page fixture
- if your test needs multiple tabs but same session /context ---------->context fixture
- if your test needs multiple user/multiple tabs/multiple sessions ----->browser fixture

ANNOTATIONS:

-In Playwright, annotations are used to add additional information or metadata to test cases. They help control test behavior, execution flow, and test categorization without changing the core test logic.

-Annotations are commonly used to describe the purpose of a test, mark tests for special execution, or temporarily skip or modify test behavior.

- Annotations help organize and manage test cases.
- They improve test readability and maintainability.

-Playwright provides built-in annotations such as skipping a test, marking a test as slow, or focusing on specific tests. These annotations are especially helpful when debugging tests or managing test execution in different environments.

-Using annotations allows testers to control how and when tests run, making test execution more flexible and efficient.

SKIP:

In Playwright, skip is an annotation used to exclude a test from execution. When a test is marked as skipped, Playwright does not run that test and reports it as skipped in the test results.

The skip annotation is commonly used when a test is not ready, a feature is under development, or there is a known issue in the application. It allows the remaining test cases to execute without interruption.

- Skipped tests are not executed.
- They appear as skipped in the test report.
- Test logic remains unchanged.

Using skip helps manage test execution effectively without deleting or commenting out test code. It is especially useful in CI pipelines and during debugging or maintenance phases.

Example:

```
test.skip('Login test', async ({ page }) => {
  // This test will not run
});
```

ONLY:

In Playwright, only is an annotation used to run only a specific test or test block. When a test is marked with only, Playwright ignores all other tests and executes only the marked one.

The only annotation is mainly used during debugging or test development when a tester wants to focus on a single test without running the entire test suite.

- Only the marked test is executed.

- All other tests are skipped automatically.
- It is useful for faster debugging.

The only annotation should be used temporarily. It is not recommended to keep only in committed code, especially in CI environments, as it can prevent other tests from running.

Example:

```
test.only('Signup test', async ({ page }) => {
  // Only this test will run
});
```

FAIL:

In Playwright, fail is an annotation used to mark a test that is expected to fail. When a test is marked with fail, Playwright treats a failure as an expected outcome instead of reporting it as an error.

The fail annotation is useful when a known defect exists in the application and the test is written to capture that behavior. Once the issue is fixed, the annotation should be removed.

- The test is expected to fail.
- A failure does not break the test run.
- If the test passes unexpectedly, it is reported.

Using fail helps teams track known issues without disabling test coverage. It allows tests to remain in the suite while clearly indicating unstable or defective functionality.

Example:

```
test.fail('Known bug test', async ({ page }) => {
  // Test is expected to fail
});
```

FIXME:

In Playwright, fixme is an annotation used to mark a test as incomplete or temporarily disabled. It indicates that the test needs to be fixed or completed in the future.

When a test is marked with fixme, Playwright skips its execution and reports it as a fixme test. This is commonly used when a test is written but not ready to run due to missing functionality or pending changes.

- The test is not executed.
- It indicates work is pending on the test.
- It appears separately in test reports.

Using fixme helps teams track unfinished or unstable tests without removing them from the test suite. Once the test is completed or the issue is resolved, the annotation should be removed.

Example:

```
test.fixme('Incomplete test', async ({ page }) => {
  // Test is not ready yet
});
```

DESCRIBE:

In Playwright, describe is used to group related test cases together. It helps organize tests based on features, modules, or functionalities, making the test suite easier to read and manage.

A describe block allows you to apply common behavior or configuration to all tests inside it. This is especially useful in large test suites where multiple tests share similar setup or purpose.

- Groups related test cases.
- Improves test readability and structure.
- Helps manage large test suites.

Annotations such as skip, only, or fail can also be applied at the describe level. When used this way, the annotation affects all tests inside that block.

Using describe is a best practice for writing clean, maintainable, and well-structured Playwright test scripts.

Example:

```
test.describe('Login Module', () => {

  test('Valid login', async ({ page }) => {
    // Test case
  });

  test('Invalid login', async ({ page }) => {
    // Test case
  });

});
```

SLOW:

In Playwright, slow is an annotation used to indicate that a test is expected to take more time to execute than normal. When a test is marked as slow, Playwright automatically increases the timeout for that test.

The slow annotation is useful for tests that involve complex workflows, multiple page navigations, or heavy data processing.

- Increases the timeout for the test.
- Prevents timeout failures for long-running tests.
- Helps manage tests with known performance delays.

The slow annotation can be applied to a single test or to a describe block. It should be used only when necessary and not as a replacement for inefficient test design.

Example:

```
test.slow('Slow running test', async ({ page }) => {
  // Timeout is increased automatically
});
```

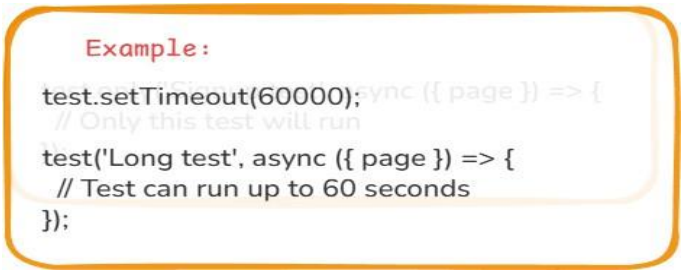
SET TIMEOUT:

In Playwright, timeout is used to define the maximum time a test, action, or assertion is allowed to take before it fails. If the operation does not complete within the specified time, Playwright throws a timeout error.

Timeouts are important when dealing with slow pages, network delays, or long-running operations. Playwright provides default timeouts, but they can be increased when required.

- Timeout is measured in milliseconds.
- It can be set globally or for specific tests or actions.
- Helps avoid indefinite waiting during test execution.

Setting proper timeouts ensures stable test execution and prevents unnecessary test failures caused by slow application behavior.



COMMANDS TO RUN PLAYWRIGHT TEST

1- npx playwright test

it will execute all the test blocks which is present in the particular project folder

2-npx playwright test "path of the file"

-it will execute the particular test file and all the test blocks present inside the particular file

-change the backward slash to forward slash

3-npx playwright test --headed

by deafult all the test cases will execute in headless mode so to execute in headed we will use this command

4- npx playwright test -g "title"

-it will the test block which have the exact or partial title

5- npx playwright test --last-failed

-this will execute the last failed test block

6-npx playwright test --project=browsername

it will execute the test block in the particular browser

7-npx playwright show-report

-it will give the report for the last test executed

8-npx playwright test --debug

it will trigger the playwright inspector

until and unless we stepover it will not execute the next line of our test block

BROWSER CONTROLS:

Browser controls in Playwright are used to launch, manage, and close the browser during test execution. These controls help testers decide how the browser should behave, such as running in headless mode or handling multiple contexts.

The browser is the top-level component in Playwright. From the browser, we create contexts, and from contexts, we create pages.

- Browser controls manage the browser lifecycle.
- They are mostly used during setup and teardown.
- Direct UI actions are not performed using the browser.

SET VIEWPORT SIZE:

setViewportSize() is used in Playwright to set the width and height of the browser viewport. It helps testers control how a web application is displayed during test execution.

Viewport size is important because many applications are responsive and behave differently based on screen resolution. By setting the viewport size, testers can validate UI behavior for different screen dimensions.

- It defines the visible area of the web page.
- It is measured in pixels.
- Commonly used for responsive testing.

Example:

```
await page.setViewportSize({ width: 1280, height: 720 });
```

NOTE:

setViewportSize() works on the page object and changes the viewport during runtime. It is different from setting viewport size at the browser context level, which is usually done during setup.

VIEWPORT SIZE:

In Playwright, viewportSize is used to set the browser viewport dimensions at the browser context level. It defines the width and height of the visible area before any page is opened.

This setting is commonly used during test setup to ensure that all pages open with a fixed screen resolution. It is especially useful for responsive UI testing and maintaining consistency across test runs.

- It is set while creating a browser context.
- It applies to all pages inside that context.
- Width and height are specified in pixels.

Example:

```
const context = await browser.newContext({
    viewport: { width: 1280, height: 720 }
                        });
```

Important Difference

- viewportSize() → Set before page creation (context level)
- setViewportSize() → Change viewport after page is created (page level)

TITLE():

title() is used in Playwright to retrieve the title of the current web page. The title is the text shown on the browser tab and is commonly used for validation in automation tests.

This method helps verify that the correct page is loaded after navigation or an action such as login or form submission.

- Returns the page title as a string.
- Works on the page object.
- Commonly used with assertions.

```
Example:
  const pageTitle = await page.title();
```

URL():

url() is used in Playwright to get the current URL of the web page. It helps verify that the application has navigated to the expected page.

This method is commonly used after actions like page navigation, form submission, or redirection to ensure the correct URL is loaded.

- Returns the current page URL as a string.
- Works on the page object.
- Useful for navigation validation.

```
Example:
    const currentUrl = page.url();
```

COOKIES():

In Playwright, cookies() is used to retrieve cookies from the browser context. Cookies store session-related information such as login state, user preferences, and authentication data.

Cookies are managed at the browser context level, not at the page level. This allows multiple pages within the same context to share the same cookies.

- Returns an array of cookie objects.
- Works on the browser context.
- Commonly used for session validation and debugging.

Example:

```
const cookies = await context.cookies();
```

ADD COOKIES():

In Playwright, addCookies() is used to add cookies to a browser context. It helps set session or authentication data before opening a page, allowing tests to start in a pre-authenticated state.

This method is commonly used to bypass repetitive login steps and improve test execution speed.

- Adds cookies at the browser context level.
- Accepts an array of cookie objects.
- Cookies apply to all pages in the context.

Example:

```
await context.addCookies([
  {
    name: 'session_id',
    value: '12345',
    domain: 'example.com',
    path: '/'
  }
]);
```

NOTE:

Cookies should be added before navigating to the application. Otherwise, the page may not recognize the session..

CLEAR COOKIES:

In Playwright, cookies are removed using the clearCookies() method. This method clears all cookies stored in the current browser context.

Removing cookies is useful when you want to reset the session, log out a user, or ensure that a test starts with a clean state.

- Clears all cookies from the browser context.
- Works at the context level.
- Affects all pages within the context.

Example:

```
await context.clearCookies();
```

CHROMIUM.LAUNCH():

chromium.launch() is used in Playwright to launch a Chromium browser instance. Chromium is the open-source browser engine used by Google Chrome and Microsoft Edge.

This method is generally used in custom or standalone setups. In the Playwright Test framework, browser launching is usually handled automatically using fixtures.

- Launches a Chromium browser.
- Returns a browser instance.
- Supports launch options such as headless mode.
- In place of chromium we can use firefox and webkit also.

Example:

```
const browser = await chromium.launch();
```

NEW CONTEXT():

newContext() is used in Playwright to create a new browser context. A browser context represents an isolated browser session, similar to a fresh browser profile.

Each context has its own cookies, cache, local storage, and session data. This ensures that tests run independently without sharing data.

- Creates an isolated browser session.
- Works on the browser object.
- Multiple contexts can be created from a single browser.

Example:

```
const context = await browser.newContext();
```

NEW PAGE():

newPage() is used in Playwright to create a new page (tab or window) inside a browser context. A page represents a single browser tab where user interactions take place.

Each page shares session data such as cookies and local storage with other pages in the same browser context.

- Creates a new browser tab.
- Works on the browser context.
- Used to interact with web pages.

Example:

```
const page = await context.newPage();
```

GOTO():

goto() is used in Playwright to navigate to a specified URL. It loads the given web page in the current browser tab.

This method is usually the first step in a test, as it opens the application under test before performing any actions or validations.

- Navigates to the provided URL.
- Works on the page object.
- Waits for the page to load before moving ahead.

```
Example:

await page.goto('https://example.com');
```

SCREENSHOT():

screenshot() is used in Playwright to capture a screenshot of the web page during test execution. Screenshots are helpful for debugging failures and for keeping visual evidence of test results.

Screenshots can be taken for the full page or for the visible viewport and are usually stored as image files.

- Captures the current state of the page.
- Works on the page object.
- Commonly used for debugging and reporting.

```
Example:

await page.screenshot({ path: 'page.png' });
```

CLOSE():

close() is used in Playwright to close a page, context, or browser, depending on the object it is called on. It helps release resources and properly end the test session.

Closing is important to avoid memory leaks and to ensure clean test execution.

- Can be used on page, context, or browser.
- Ends the current session or tab.
- Commonly used during teardown.

Example:

```
await browser.close();
await page.close();
```

NOTE:

When using Playwright Test framework, pages and browsers are usually closed automatically by fixtures. Manual use of close() is mainly required in custom setups..

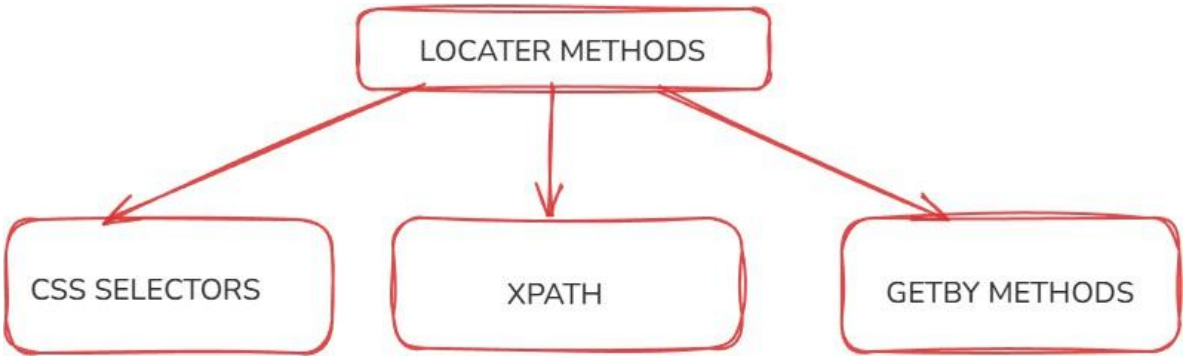
LOCATERS:

In Playwright, locators are used to identify and interact with elements on a web page. They act as a bridge between the test script and the user interface.

Locators allow Playwright to find elements in a reliable way. They automatically wait for elements to appear, become visible, and be ready for interaction before performing any action. This built-in auto-waiting helps reduce test failures caused by timing issues.

Playwright provides modern and user-focused locator strategies such as role-based, text-based, label-based, and test-id-based locators. These locators are designed to be readable, stable, and closer to real user behaviour .

Using proper locators improves test stability, readability, and maintainability, making them a fundamental part of Playwright automation.



SYNTAX:

page.locator('css-selector')

page.locator('xpath-selector')

CSS SELECTOR:

A CSS selector is used to identify HTML elements based on their tag name, id**,** class, attributes, or hierarchy. In Playwright, CSS selectors are commonly used with the locator() method to find elements on a web page.

CSS selectors are fast and flexible, but they depend on the page structure. Therefore, they should be used carefully to avoid unstable tests.

XPATH:

XPath (XML Path Language) is used to locate elements by navigating through the HTML DOM structure. In Playwright, XPath is mainly used when elements cannot be uniquely identified using GetBy methods or CSS selectors.

XPath is powerful and flexible, but it is more complex and sensitive to UI changes. Therefore, it should be used as a last option.

GETBY METHODS:

GetBy methods in Playwright are built-in locator methods used to identify web elements in a user-focused and reliable way. These methods are designed around accessibility standards and represent how real users see and interact with the application.

GetBy methods automatically handle waiting and retries. Playwright waits for the element to appear, become visible, and be ready for interaction before performing any action. This makes tests more stable and reduces flakiness.

GetBy methods are preferred over CSS and XPath because they are readable, stable, and easy to maintain. They also encourage the use of semantic HTML, which improves accessibility and automation quality.

getByLabel –

getByLabel() is a Playwright locator method used to identify form elements by their associated label text. It locates input fields, text areas, or select elements in the same way a user understands a form—through the visible label.

This method improves test readability and accessibility because it relies on proper HTML label associations rather than technical selectors.

Syntax:

page.getByLabel(labelText)

getByPlaceholder

getByPlaceholder() is a Playwright locator method used to identify input fields using the placeholder text displayed inside them. It reflects how users recognize input fields before typing.

This method is especially useful when labels are not present or not properly associated with form elements.

Syntax:

page.getByPlaceholder(placeholderText)

getByText –

getByText() is a Playwright locator method used to locate elements based on their visible text content. It identifies elements the same way a user does—by reading what is displayed on the screen.

This method is commonly used for buttons, links, headings, and messages where text is clearly visible.

Syntax:

page.getByText(text)

getByAltText

getByAltText() is a Playwright locator method used to locate elements—mainly images—using the value of their alt attribute. The alt text describes the image content and is important for accessibility.

This method is useful when interacting with images, icons, or elements where meaningful alt text is provided.

Syntax:

page.getByAltText(altText)

getByTitle –

getByTitle() is a Playwright locator method used to locate elements using the value of their title attribute. The title attribute usually appears as a tooltip when a user hovers over an element.

This method is useful when elements have meaningful title attributes and other locator options are not suitable.

Syntax:

page.getByTitle(titleText)

Example:

```
page.getByTitle(titleText)
```

getByRole() –

getByRole() is a Playwright locator method used to locate elements based on their ARIA role and accessible name. It identifies elements in the same way users and assistive technologies understand them.

This method works best with semantic HTML and is the most recommended locator in Playwright because it is reliable and readable.

Syntax:

page.getByRole(role, options)

Example:

```
page.getByRole(role, options)
```

NOTE:

ARIA stands for Accessible Rich Internet Applications.ARIA is a set of attributes used in HTML to make web applications accessible, especially for users who rely on screen readers.ARIA helps describe what an element is and how it behaves when normal HTML is not sufficient..

NOTE:

The name option is used with getByRole() to specify the accessible name of an element. It helps Playwright uniquely identify an element when multiple elements share the same role.The accessible name is the text that users and screen readers recognize for an element. In getByRole() name option is used most of the time.

getbyTestId()

Locates element using data-testid attribute/custom attributes configured as test ID.Perfect for dynamic content,elements which dont have meaningful labels,text

Syntax:

await page.getbyTestId("testid_value",{options})

It is used mainly when,(data-testid)

-it never changes in UI design

-it is not dependent on class

-it is not dependent on any visible text

-it is not dependent on DOM structure

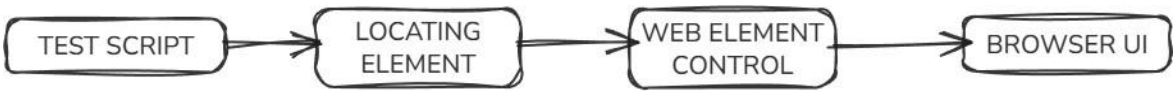
```
Example:
 page.getByTestId(testId)
```

WEB ELEMENT CONTROLS:

Web element controls in Playwright are actions performed on web elements to simulate real user behaviour. These controls allow automation scripts to interact with the application by clicking elements, entering text, selecting options, and performing mouse or keyboard actions.

Web element controls are executed on locators, not directly on elements. Before performing any action, Playwright automatically waits for the element to be visible, enabled, and stable. This built-in auto-waiting helps prevent synchronization issues and makes tests more reliable.

Web element controls are essential in automation testing because they represent how users interact with the application UI. Proper use of these controls ensures accurate test execution and stable automation scripts.



FILL():

fill() is a web element control in Playwright used to enter text into input fields such as text boxes and text areas. Before entering new text, it automatically clears any existing value in the field.

The fill() method works on a locator and waits for the element to be visible, enabled, and editable before performing the action. This makes it reliable and safe to use in automation tests.

Example:

```
await page.getByLabel('Username').fill('admin');
```

Important Points

-Clears existing text before typing
-Works only on editable elements
-Includes auto-waiting and retries
-Preferred for form input automation

TYPE():

type() is a web element control in Playwright used to enter text into input fields by simulating real keyboard typing. Unlike fill(), it does not clear existing tex**t** before entering new characters.

The type() method works on a locator and waits for the element to be visible, enabled, and editable before performing the action.

Example:

```
await page.getByLabel('Search').type('Playwright');
```

Important Points

-Appends text to existing content
-Simulates real keyboard input
-Slower than fill()
-Useful when testing key-by-key behavior

innerText():

innerText() is used in Playwright to retrieve the visible text content of an element as it appears on the UI. It returns only the text that is rendered and visible to the user.

This method ignores hidden text and reflects the actual text shown on the screen, making it useful for UI validations.

Example:

```
const text = await page.getByRole('heading').innerText();
```

Important Points

-Returns only visible text
-Ignores hidden or CSS-hidden content
-Affected by styling and layout
-Commonly used for validation

textContent():

textContent() is used in Playwright to retrieve all text content of an element, including text that is not visible on the UI. It returns the raw text present in the DOM.

Unlike innerText(), textContent() is not affected by CSS styles or visibility.

Example:

```
const text = await page.locator('#message').textContent();
```

Important Points

-Returns visible and hidden text
-Not affected by CSS or layout
-Faster than innerText()
-Used for DOM-level validation

CLICK():

click() is a web element control in Playwright used to simulate a mouse click on an element such as a button, link, checkbox, or icon.

The click() method works on a locator and automatically waits for the element to be visible, enabled, and stable before performing the action. This ensures reliable interaction with the UI.

Example:

```
await page.getByRole('button', { name: 'Submit' }).click();
```

Important Points

-Performs a left mouse click
-Works only on visible and enabled elements
-Includes auto-waiting and retry mechanism
-Triggers associated UI events

inputValue():

inputValue() is used in Playwright to retrieve the current value of an input or textarea element. It returns the value exactly as it is present in the input field.

This method is useful for validating entered data or default values in form fields.

Example:

```
const value = await page.getByLabel('Username').inputValue();
```

Important Points

-Works only on input and textarea elements
-Returns the current value of the field
-Useful for form validation
-Does not retrieve visible text outside inputs

allTextContents():

allTextContents() is used in Playwright to retrieve text content from all elements matched by a locator. It returns an array of strings, where each string represents the text content of one matched element.

This method is useful when multiple elements share the same locator and you want to collect their text values together.

Example:

```
const texts = await page.locator('.product-name').allTextContents();
```

Important Points

-Returns an array of text values
-Includes visible and hidden text (DOM-level)
-Works when multiple elements match the locator
-Useful for list and table validations

allInnerTexts():

allInnerTexts() is used in Playwright to retrieve visible text from all elements matched by a locator. It returns an array of strings, where each string represents the text that is actually displayed to the user.

Unlike allTextContents(), this method returns only visible text and ignores hidden content.

Example:

```
const texts = await page.locator('.menu-item').allInnerTexts();
```

Important Points

-Returns an array of visible text values
-Ignores hidden or CSS-hidden text
-Reflects what the user sees on the UI
-Useful for validating lists and menus

getAttribute():

getAttribute() is used in Playwright to retrieve the value of a specified attribute from a web element. Attributes provide additional information about elements, such as id, class, href, src, or custom attributes.

This method works on a locator and returns the attribute value as a string.

Example:

```
const link = await page.getByRole('link', { name: 'Home' }).getAttribute('href');
```

Important Points

-Returns the value of the specified attribute
-Returns null if the attribute is not present
-Works for standard and custom attributes
-Useful for validation and debugging

all():

all() is a Playwright locator method used to retrieve all elements matched by a locator as an array of Locator objects. It allows you to work with multiple elements individually when the same locator matches more than one element.

This method is useful when you need to iterate over elements and perform actions or validations on each one separately.

Example:

```
const items = await page.locator('.menu-item').all();

for (const item of items) {

console.log(await item.innerText());

}
```

Important Points

-Returns an array of Locator objects, not text
-Each locator can be acted upon individually
-Useful for loops and conditional checks
-Works when multiple elements match the locator

waitFor():

waitFor() is used in Playwright to explicitly wait for a specific condition on a locator before continuing test execution. It ensures that the element reaches the required state, such as becoming visible, hidden, attached, or detached from the DOM.

Although Playwright has built-in auto-waiting, waitFor() is useful in special scenarios where you need precise control over synchronization.

Example:

```
await page.getByText('Success').waitFor({ state: 'visible' });
```

Common States:

- visible – waits until the element is visible
- hidden – waits until the element is hidden
- attached – waits until the element is present in the DOM
- detached – waits until the element is removed from the DOM

Important Points

-Works on a locator
-Used for explicit waiting
-Should be used only when necessary
-Not a replacement for Playwright auto-waits

isVisible():

isVisible() is used in Playwright to check whether a web element is visible on the page. It returns a boolean value—true if the element is visible to the user, otherwise false.

This method does not wait for the element to become visible. It immediately checks the current state of the element.

Example:

```
const status = await page.getByText('Welcome').isVisible();
```

isEnabled():

isEnabled() is used in Playwright to check whether a web element is enabled and ready for interaction. It returns a boolean value—true if the element is enabled, otherwise false.

An enabled element is one that is not disabled and can accept user actions like click or input.

Example:

```
const status = await page.getByRole('button', { name: 'Submit' }).isEnabled();
```

isDisabled():

isDisabled() is used in Playwright to check whether a web element is disabled. It returns a boolean value—true if the element is disabled, otherwise false.

A disabled element cannot be interacted with and usually has the disabled attribute.

Example:

```
const status = await page.getByRole('button', { name: 'Submit' }).isDisabled();
```

isEditable():

isEditable() is used in Playwright to check whether an element is editable. It returns a boolean value—true if the element can be edited by the user, otherwise false.

An element is considered editable when it is visible, enabled, and not read-only.

Example:

```
const status = await page.getByLabel('Username').isEditable();
```

isChecked():

isChecked() is used in Playwright to check whether a checkbox or radio button is selected. It returns a boolean value—true if the element is checked, otherwise false.

This method is applicable only to checkboxes and radio buttons.

Example:

```
const status = await page.getByRole('checkbox', { name: 'Accept Terms' }).isChecked();
```